

PROJECT PRESENTATION

BudgetBuddy AI

An AI-powered expense tracker built with Firebase,
FastAPI and Groq

Anushay Ayat

AI and Data Science | SMIT

Why BudgetBuddy AI?



Hard to keep up

Notes and spreadsheets are easy to start and quickly abandoned.



Local habits ignored

JazzCash, Easypaisa, bank SMS alerts and Roman Urdu notes are everyday reality.



Privacy and secrets

Financial data must stay private and API keys must never reach the browser.

Goal: record spending in seconds, in your own words, and get answers from your own data.

What it does



Expense tracking

Form, table, edit, delete with Undo, live totals



AI quick add

Type or paste an SMS; AI fills the form



Ask your spending

Plain-language questions about your own data



Budgets and charts

6-month chart, forecast, custom categories



Wallets and bills

Income, wallets, recurring bills, bill splitting



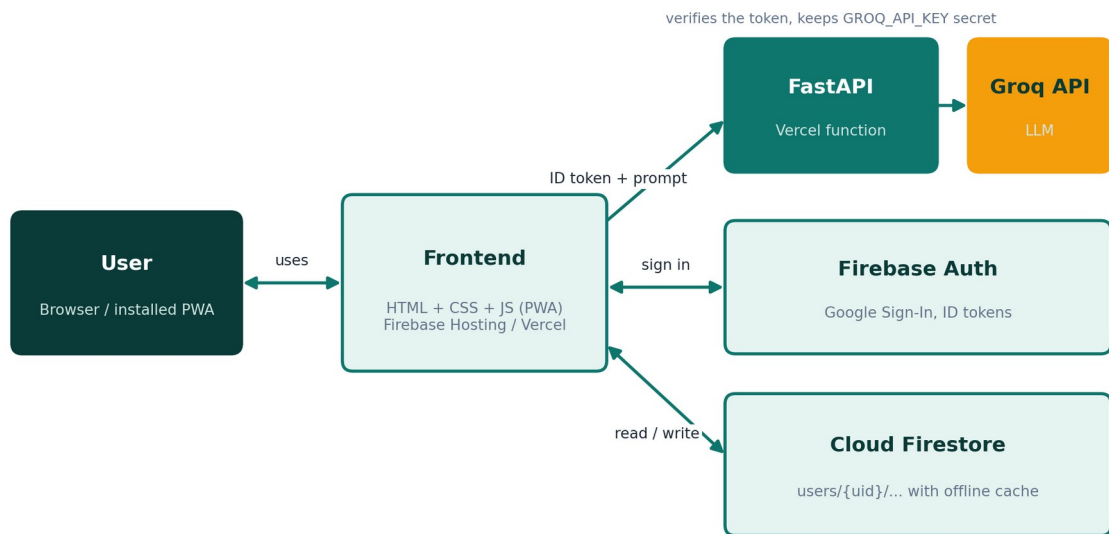
Offline and Urdu

Installable PWA, English and Urdu (RTL)

Assignment requirements: all met

Requirement	Implementation	Status
AI Expense Tracker using Groq	AI page via FastAPI /api/ai	Done
Form: Title, Amount, Category, Date, Add	Five required categories + custom ones	Done
Store records in Firestore	users/{uid}/expenses	Done
Table of saved expenses below form	Real-time listener	Done
Delete button	With one-tap Undo	Done
Total count and total amount	Update instantly	Done
Deploy and share live link	Firebase Hosting + Vercel	Done

System architecture



1

Frontend

Static site on Firebase Hosting or Vercel

2

Firebase

Google sign-in and Firestore data per user

3

FastAPI

Verifies token, calls Groq with the secret key

How an AI request is protected



Key stays on the server

GROQ_API_KEY is an environment variable



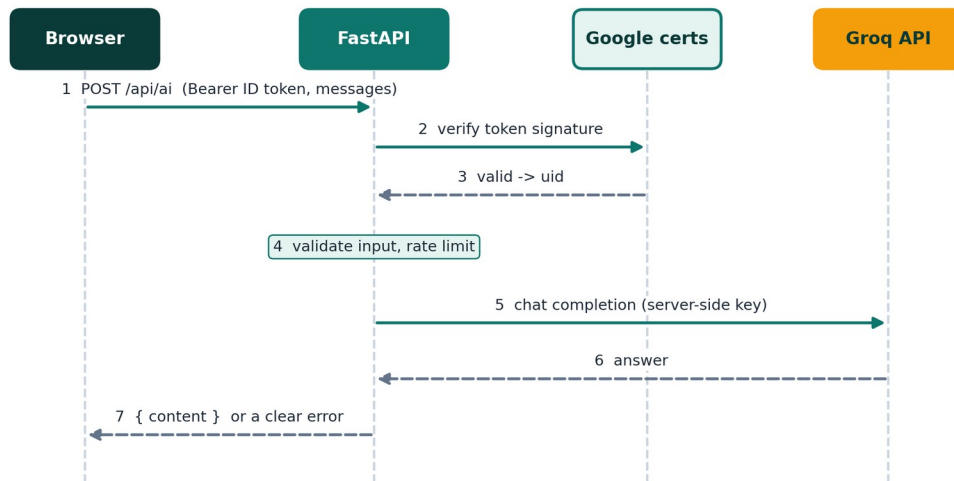
Only signed-in users

Firebase ID token is verified on every call



Validated and limited

Roles, sizes, 30 requests per minute



Technology stack



Frontend

- HTML5, CSS3
- Vanilla JavaScript
- SVG charts
- Service Worker (PWA)



Backend

- Python, FastAPI
- Pydantic, httpx
- google-auth
- pytest



Firebase

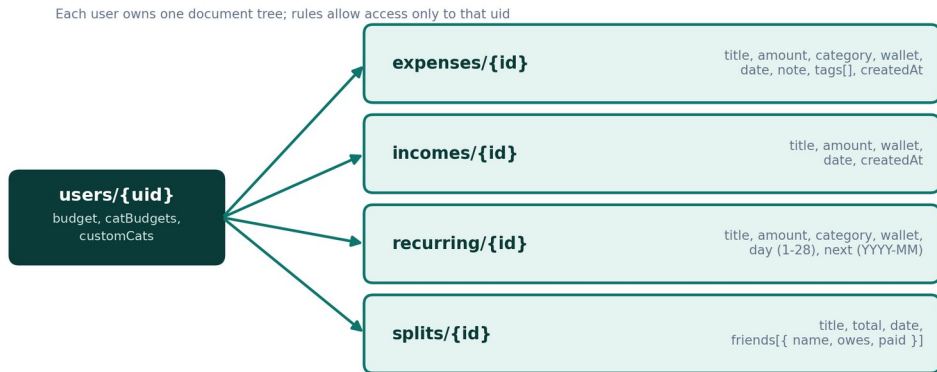
- Authentication (Google)
- Cloud Firestore
- Security rules
- Hosting



AI and hosting

- Groq LLM API
- Openai/gpt-oss-20b
- Vercel serverless
- GitHub

Data model and privacy



Each user owns one tree of documents.
Nothing is shared between users.

```
firebase.rules  
match /users/{uid}/**  
  allow read, write:  
    if request.auth.uid  
      == uid;
```

- Offline cache and sync
- Real-time listeners
- Escaped user text
- Custom categories in customCats

AI features and prompt design



Quick add

Free text or bank SMS becomes a pre-filled form



Ask your spending

Answers only from your own data



Auto-category

Suggests one of your categories



Monthly review

Overview, 2 highlights, 2 suggestions

Prompt design

- JSON mode with fixed allowed values
- Grounded: answer only from supplied data
- Titles and notes treated as data, not instructions
- AI fills the form; the user confirms

Custom categories



Create anywhere

New category... in any form, or Manage categories



Name and colour

Up to 24 characters, 20 per user, no duplicates



Works everywhere

Charts, limits, filters, reports and AI



Safe delete

Expenses move to Other; the limit is removed

How it is stored

- Saved as `users/{uid}.customCats`
- Same security rule; no rule change
- Names escaped, colours validated
- 29 automated jsdom checks pass

Backend API and testing

FastAPI endpoints

Route	Purpose
GET /api/health	Status, key configured, model
POST /api/ai	Groq chat, needs Firebase token

- 1 to 6 messages, 12,000 characters each
- 30 requests per minute per user
- CORS limited to allowed origins
- Groq errors mapped to clear messages

17 / 17

pytest tests passing

- Validation and error mapping
- Firebase token checks
- Rate limit and CORS
- Mocked Groq: no real calls

Deployment

1

GitHub

Push the project to a repository

2

Vercel

Backend + frontend. Add 3 environment variables, deploy, open /api/health

3

Firebase Hosting

Set API_BASE to the Vercel URL, then firebase deploy

Environment variables

GROQ_API_KEY

FIREBASE_PROJECT_ID

ALLOWED_ORIGINS

Limitations and future work

Current limitations

- Rate limiter is per instance (in memory)
- Recurring bills created when the app opens
- Voice input needs Chrome or Edge
- Equal-share bill splitting only
- Frontend mostly tested manually
- Categories cannot be renamed yet

Next steps

- Persistent rate limiting (Redis or Firestore)
- Scheduled recurring bills and reminders
- Receipt photo upload
- Unequal splits, editable recurring rules
- End-to-end frontend tests
- Rename and re-colour categories

Thank you

Questions?

Firebase <https://budgetbuddy-ai-23447.web.app>

Vercel <https://budgetbuddy-ai-xi.vercel.app>

GitHub <https://github.com/scientistayat-engineer/BudgetBuddy-AI>